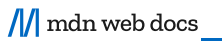


MDN Plus now available in [your](#) country! Support MDN [and](#) make it your own. [Learn more](#) ✨



This page was translated from English by the community. [Learn more](#) and [join the MDN Web Docs community](#).

Array.prototype.sort()

El método **sort()** ordena los elementos de un arreglo (array) *localmente* y devuelve el arreglo ordenado. La ordenación no es necesariamente [estable](#) . El modo de ordenación por defecto responde a la posición del valor del string de acuerdo a su valor [Unicode](#) .

La fuente de este ejemplo interactivo se almacena en un repositorio de GitHub. Si desea contribuir al proyecto de ejemplos interactivos, clone <https://github.com/mdn/interactive-examples> y envíenos una solicitud de extracción (pull request).

Sintaxis

```
arr.sort([compareFunction])
```

Parámetros

compareFunction

Opcional. Especifica una función que define el modo de ordenamiento. Si se omite, el array es ordenado atendiendo a la posición del valor [Unicode \(en-US\)](#) de cada caracter, según la conversión a string de cada elemento.

firstEl

El primer elemento a comparar.

secondEl

El segundo elemento a comparar.

Valor devuelto

El array ordenado.

Descripción

Si no se provee **compareFunction** , los elementos son ordenados convirtiéndolos a strings y comparando la posición del valor Unicode de dichos strings. Por ejemplo, "Cherry" viene antes que "banana" (porque las mayúsculas van antes que las minúsculas en la codificación Unicode) . En un ordenamiento numérico, 9 está antes que 80, pero dado que los números son convertidos a strings y ordenados según el valor Unicode, el resultado será "80" antes que "9".

```
var frutas = ['guindas', 'manzanas', 'bananas'];
frutas.sort(); // ['bananas', 'guindas', 'manzanas']
```

```
var puntos = [1, 10, 2, 21];
puntos.sort(); // [1, 10, 2, 21]
// Tenga en cuenta que 10 viene antes que 2
// porque '10' viene antes que '2' según la posición del valor Unicode.
```

```
var cosas = ['word', 'Word', '1 Word', '2 Words'];
cosas.sort(); // ['1 Word', '2 Words', 'Word', 'word']
// En Unicode, los números vienen antes que las letras mayúsculas
// y estas vienen antes que las letras minúsculas.
```

Si se provee `compareFunction`, los elementos del array son ordenados de acuerdo al valor que retorna dicha función de comparación. Siendo `a` y `b` dos elementos comparados, entonces:

- Si `compareFunction(a, b)` es menor que 0, se sitúa `a` en un índice menor que `b`. Es decir, `a` viene primero.
- Si `compareFunction(a, b)` retorna 0, se deja `a` y `b` sin cambios entre ellos, pero ordenados con respecto a todos los elementos diferentes. Nota: el estándar ECMAScript no garantiza este comportamiento, por esto no todos los navegadores (p.ej. Mozilla en versiones que datan hasta el 2003) respetan esto.
- Si `compareFunction(a, b)` es mayor que 0, se sitúa `b` en un índice menor que `a`.
- `compareFunction(a, b)` siempre debe retornar el mismo valor dado un par específico de elementos `a` y `b` como sus argumentos. Si se retornan resultados inconsistentes entonces el orden de ordenamiento es indefinido.

Entonces, la función de comparación tiene la siguiente forma:

```
function compare(a, b) {
  if (a es menor que b según criterio de ordenamiento) {
    return -1;
  }
  if (a es mayor que b según criterio de ordenamiento) {
    return 1;
  }
  // a debe ser igual b
  return 0;
}
```

Para comparar números en lugar de strings, la función de comparación puede simplemente restar `b` de `a`. La siguiente función ordena el array de modo ascendente:

```
function compareNumbers(a, b) {
  return a - b;
}
```

El método `sort` puede ser usado convenientemente con [function expressions](#) (y [closures](#)):

```
var numbers = [4, 2, 5, 1, 3];
numbers.sort(function(a, b) {
  return a - b;
});
console.log(numbers); // [1, 2, 3, 4, 5]
```

Los objetos pueden ser ordenados por el valor de una de sus propiedades.

```
var items = [
  { name: 'Edward', value: 21 },
  { name: 'Sharpe', value: 37 },
  { name: 'And', value: 45 },
  { name: 'The', value: -12 },
```

```
{ name: 'Magnetic', value: 13 },
{ name: 'Zeros', value: 37 }
];
items.sort(function (a, b) {
  if (a.name > b.name) {
    return 1;
  }
  if (a.name < b.name) {
    return -1;
  }
  // a must be equal to b
  return 0;
});
```

Ejemplos

Ordenando un array

Un array de elementos string, sin especificar una función de comparación:

```
var arr = [ 'a', 'b', 'Z', 'Aa', 'AA' ];
arr.sort(); // [ 'AA', 'Aa', 'Z', 'a', 'b' ]
```

Un array de elementos numéricos, sin función de comparación:

```
var arr = [ 40, 1, 5, 200 ];
arr.sort(); // [ 1, 200, 40, 5 ]
```

Un array de elementos numéricos, usando una función de comparación:

```
var arr = [ 40, 1, 5, 200 ];
function comparar ( a, b ){ return a - b; }
arr.sort( comparar ); // [ 1, 5, 40, 200 ]
```

Lo mismo pero usando una función anónima normal:

```
var arr = [ 40, 1, 5, 200 ];
arr.sort(function(a,b){return a - b;}); // [ 1, 5, 40, 200 ]
```

Lo mismo escrito más compacto mediante una [función flecha](#):

```
var arr = [ 40, 1, 5, 200 ];
arr.sort((a,b)=>a-b); // [ 1, 5, 40, 200 ]
```

Creando, mostrando, y ordenando un array

El siguiente ejemplo abunda en la idea de ordenar con y sin función de comparación. Además, ilustra una manera de mostrar un array una vez creado. El método [join](#) es usado para convertir el array en una cadena de texto que imprimir. Al no pasarle un argumento que indique el separador, usará la coma por defecto para separar los elementos del array dentro de la cadena.

```
var arr = ['80', '9', '700', 40, 1, 5, 200];
function comparar(a, b) {
  return a - b;
```

```
}  
console.log('original:', arr.join());  
console.log('ordenado sin función:', arr.sort());  
console.log('ordenado con función:', arr.sort(comparar));
```

El ejemplo produce el siguiente resultado. Como muestra la salida, cuando una función de comparación es usada, los números se ordenan correctamente, sean estos valores numéricos o strings numéricos.

```
original: 80,9,700,40,1,5,200  
ordenado sin función: 1,200,40,5,700,80,9  
ordenado con función: 1,5,9,40,80,200,700
```

Ordenando caracteres no ASCII

Para ordenar strings con caracteres no ASCII, i.e. strings con caracteres con acento (e, é, è, a, ä, etc.), strings de lenguajes diferentes al inglés: use [String.localeCompare](#). Esta función puede comparar esos caracteres para que aparezcan en el orden correcto.

```
var items = ['réservé', 'premier', 'cliché', 'communiqué', 'café', 'adieu'];  
items.sort(function (a, b) {  
    return a.localeCompare(b);  
});  
  
// items is ['adieu', 'café', 'cliché', 'communiqué', 'premier', 'réservé']
```

Ordenando con map

La `compareFunction` puede ser invocada múltiples veces por elemento dentro del array. Dependiendo de la naturaleza de `compareFunction`, este puede resultar en una alta penalización de rendimiento. Cuanto más trabajo hace una `compareFunction` y más elementos hay para ordenar, resulta más recomendable usar una función [map](#) para ordenar. La idea es recorrer el array una sola vez para extraer los valores usados para ordenar en un array temporal, ordenar el array temporal y luego recorrer el array para lograr el orden correcto.

```
// el array a ordenar  
var list = ['Delta', 'alpha', 'CHARLIE', 'bravo'];  
  
// array temporal contiene objetos con posición y valor de ordenamiento  
var mapped = list.map(function(el, i) {  
    return { index: i, value: el.toLowerCase() };  
});  
  
// ordenando el array mapeado que contiene los valores reducidos  
mapped.sort(function(a, b) {  
    if (a.value > b.value) {  
        return 1;  
    }  
    if (a.value < b.value) {  
        return -1;  
    }  
    return 0;  
});  
  
// contenedor para el orden resultante  
var result = mapped.map(function(el){  
    return list[el.index];  
});
```

Especificaciones

Especificación	Estado	Comentario
ECMAScript 1st Edition	Standard	Definición inicial.
ECMAScript 5.1 (ECMA-262) La definición de 'Array.prototype.sort' en esta especificación.	Standard	
ECMAScript 2015 (6th Edition, ECMA-262) La definición de 'Array.prototype.sort' en esta especificación.	Standard	

Compatibilidad en navegadores

[Report problems with this compatibility data on GitHub](#)

	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	
sort	Chrome 1	Edge 12	Firefox 1	Opera 4	Safari 1	Chrome 18 Android	Firefox 4 for Android	Opera 10.1 Android	Sa
Stable sorting	Chrome 70	Edge 79	Firefox 3	Opera 57	Safari 10.1	Chrome 70 Android	Firefox 4 for Android	Opera 49 Android	Sa c iC

Tip: you can click/tap on a cell for more information.

Full support

Compatibilidad en navegadores

La tabla de compatibilidad en esta página es generada por una data estructurada. Si deseas contribuir a la data, por favor entra a <https://github.com/mdn/browser-compat-data> y envíanos un pull request.

Ver también

- [Array.prototype.reverse\(\)](#)
- [String.prototype.localeCompare\(\)](#)

Last modified: 29 sept 2022, [by MDN contributors](#)